Учебный практикум по имитационному моделированию

Система автоматизации функций секретаря

Отчёт о разработке программной системы

Реализация на C++17 и Qt Widgets

Версия модели: `calendar-v7`.
Основание: общая постановка практикума и вариант «Система автоматизации функций секретаря».
Состояние документации: 6 ноября 2026 года.
Учебное заведение, группа и данные автора заполняются перед сдачей.

2026

Содержание

1 Уточнённая постановка задачи **3**

1.1 Назначение и исходные требования . . . 3

1.2 Принятые проектные решения . . . 3

1.3 Роли и границы ручного управления . . . 4

1.4 Существенное ограничение относительно задания . . . 4

1.5 Исходные материалы . . . 4

2 Объектно-ориентированное проектирование и диаграммы **5**

2.1 Объектный анализ . . . 5

2.2 Диаграмма объектов . . . 7

2.3 Правила модельного времени и актуализации . . . 8

2.4 Подбор разрешений конфликта . . . 9

3 Текстовые спецификации основных классов **9**

3.1 Общие соглашения интерфейсов . . . 9

3.2 EventData и CalendarEvent . . . 9

3.3 CalendarSetup . . . 10

3.4 CalendarService . . . 10

3.5 SimulationEngine . . . 11

3.6 ConflictResolver . . . 13

3.7 ReminderService и канал уведомлений . . . 13

3.8 SimulationReport . . . 14

3.9 SimulationStore и JsonCodec . . . 14

3.10 ExperimentRunner . . . 15

3.11 Основные классы интерфейса . . . 15

4 Использованные инструментальные средства **16**

5 Файловая и модульная структура **17**

6 Краткая характеристика пользовательского интерфейса **19**

6.1 Задание параметров . . . 19

6.2 Окно моделирования . . . 19

6.3 Добавление, редактирование и конфликты . . . 20

6.4 Почта, отчёт и исследование . . . 20

7 Проведённые эксперименты и исследование модели **20**

7.1 Цель и схема исследования 20
7.2 Статистические факторы и показатели 21
7.3 Фактические результаты 22
7.4 Ограничения интерпретации 22

8 Подготовка Git-репозитория **23**

A Проверка реализации и воспроизводимость **23**
A.1 Автоматические проверки 23
A.2 Воспроизведение исследования и сборка отчёта 24
A.3 Соответствие пунктам отчёта из задания 25

1 Уточнённая постановка задачи

1.1 Назначение и исходные требования

Разработать объектно-ориентированную программную систему, автоматизирующую ведение общего календаря фирмы и позволяющую исследовать актуализацию календаря при пошаговом изменении модельного времени. Исходная постановка требует визуализации с движущимися объектами, задания параметров перед экспериментом и статистического моделирования неопределённых факторов.

Фирма состоит из K отделов, $5 \leq K \leq 9$. Во внутреннем календаре представлены ежедневные планёрки отделов, еженедельные совещания руководителей, командировки, тематические семинары и другие встречи. У мероприятия имеются название, тип, начало и окончание, участники, организатор, место и важность. Внутренние мероприятия используют помещения отделов или конференц-зал. Командировка не занимает внутреннее помещение.

Пользователь добавляет новые планы, уточняет существующие и получает сведения о событиях за день или период дня по фирме, отделу и сотруднику. Проверяется одновременное участие сотрудника в разных мероприятиях и пересечение бронирований помещения. Выявленный конфликт должен быть объяснён пользователю; желателен автоматический подбор нескольких способов его разрешения. Возможность изменения существующих планов зависит от категории пользователя.

Период опыта составляет N календарных дней, $7 \leq N \leq 30$, шаг — 30 или 60 минут. По завершении формируется отчёт о событиях фирмы за период. Уведомления участникам планируются по важности мероприятий.

1.2 Принятые проектные решения

В текущем проекте реализовано локальное приложение. Состав отделов, сотрудников и помещений постоянен на протяжении опыта. В каждом отделе первоначально руководителем является первый сотрудник. Отдельные кадровые и психологические процессы не входят в модель.

При включённом начальном календаре все рабочие дни периода заполняются 30-минутными планёрками каждого отдела. Они начинаются одновременно в начале рабочего дня, но проходят в разных помещениях и не конфликтуют. В первый рабочий день каждой календарной недели внутри периода после планёрок назначается 30-минутное совещание руководителей в конференц-зале. При выключенном пресете календарь первоначально пуст. Пользователь может дополнить его до продвижения времени.

Периодические семинары имеют отдельный шаблон и создаются во время прогона с указанной заблаговременностью. Их участники представляют минимум два отдела. Если дата нерабочая, она переносится вперёд. Случайные заявки и попытки изменения расписания генерируются независимо от визуальной скорости.

Модельное время хранится с точностью до минуты, со смещением UTC+03:00. Фактические часы компьютера не определяют ход событий. Начатые и завершённые мероприятия нельзя редактировать; завершённые исключаются из активного календаря, сохраняясь в архиве и истории.

1.3 Роли и границы ручного управления

До запуска выбирается роль оператора. В текущем интерфейсе оператор связан с сотрудником с внутренним ID 1; выбор произвольной учётной записи не реализован. Роль — условие учебной модели, а не сетевая аутентификация.

- Секретарь может изменять весь календарь.
- Руководитель может редактировать планы в разрешённом отделе. Проверяются организатор, участники и принадлежность помещения до и после изменения.
- Сотрудник может создать мероприятие от своего имени, но не изменяет уже назначенные события.

Добавление, редактирование и выбор варианта переноса ставят прогон на паузу. Продолжение выполняется явно. Переводы сотрудников, назначения руководителей и отдельная кнопка отмены мероприятия в интерфейсе отсутствуют. Операция отмены сохранена в календарном сервисе для внутренних сценариев и проверок.

1.4 Существенное ограничение относительно задания

Электронная почта имитируется. Программа рассчитывает сроки и адресатов, сохраняет тему, текст, момент отправки и доставки, но не отправляет сообщения через SMTP. Модель считает доставку мгновенной и успешной. Поэтому требование настоящей электронной рассылки полностью не выполнено; применение имитации необходимо согласовать с преподавателем либо дополнить канал реальной отправкой. В настоящем отчёте эти действия не называются реальной доставкой.

1.5 Исходные материалы

Постановка уточнена по двум предоставленным документам: «Практикум. Моделирование. Общая постановка задачи» и «Система автоматизации функций секретаря». Сведения о реализации сверены с заголовками и исходниками текущего проекта, а результаты исследования — с полными записями прогонов calendar-v7.

2 Объектно-ориентированное проектирование и диаграммы

2.1 Объектный анализ

Предметные сущности — отдел, сотрудник, помещение и мероприятие. Данные редактируемого плана выделены в `EventData`. `CalendarEvent` добавляет идентификатор, статус, версии и снимки сведений об участниках. Изменение данных отделено от отображения: графический объект сотрудника связан с моделью по ID, но не управляет её временем.

`CalendarService` хранит календарь.

`SimulationEngine` задаёт ход эксперимента. Планирование уведомлений, подбор переносов, построение отчёта и сохранение представлены отдельными классами. Композиция используется для объектов одного прогона. Наследование введено в канале уведомлений: абстрактный `NotificationChannel` реализуется классом `SimulatedEmailChannel`. Классы интерфейса наследуются от `QWidget`, `QDialog` или `QObject`.

На диаграммах сплошная линия с ромбом показывает композицию, пунктирная стрелка — использование, пустой треугольник — наследование. В диаграммах перечислены основные поля и операции, а не весь интерфейс Qt и вспомогательные типы.

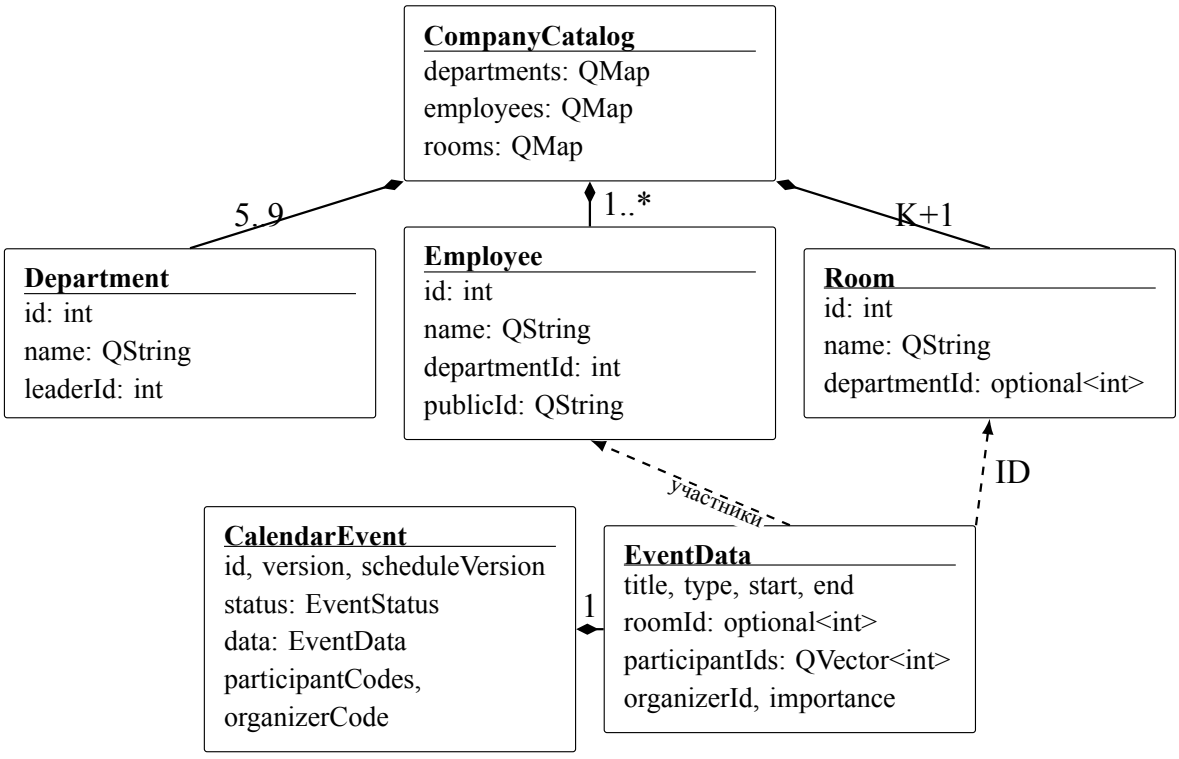
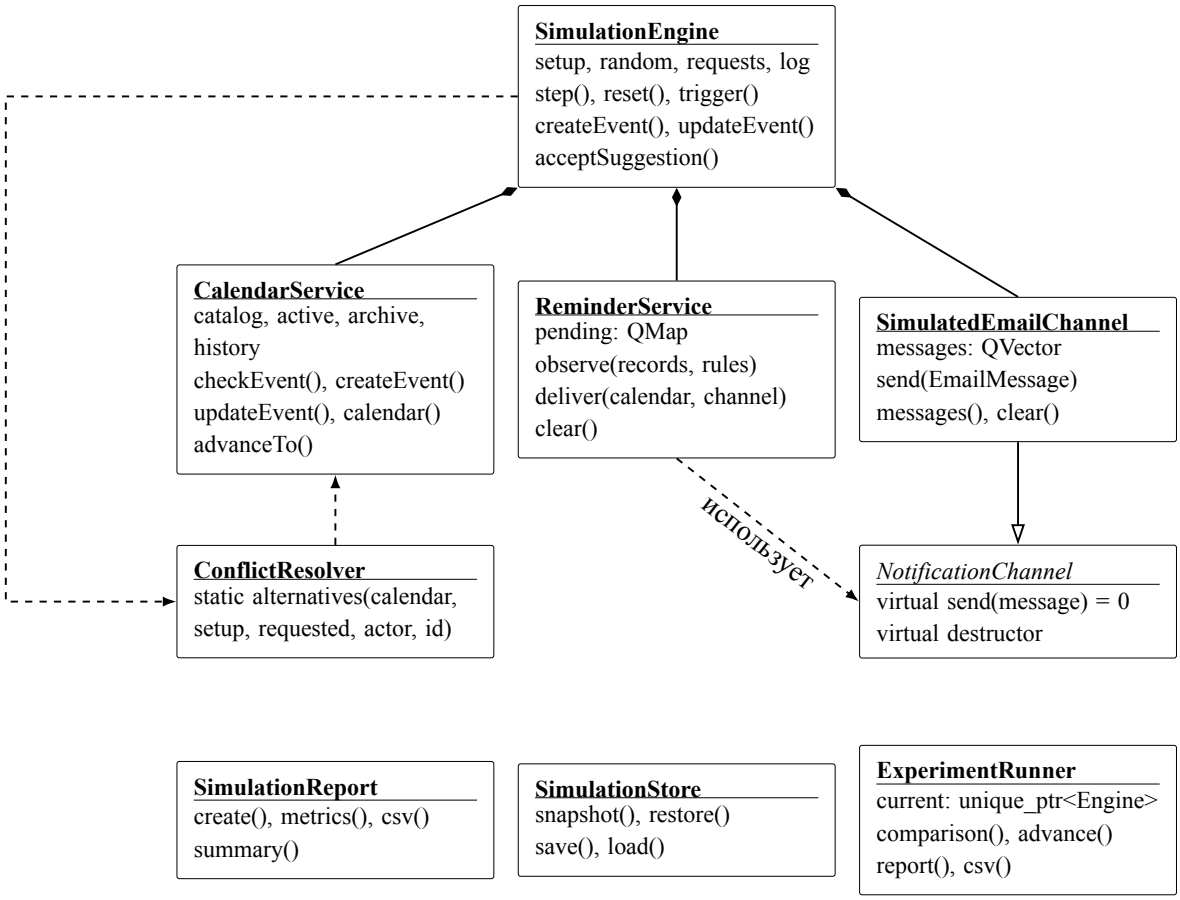


Рис. 1: Диаграмма классов предметной области. Ссылки на сотрудников и помещения хранятся как ID.



SimulationReport читает движок; SimulationStore воспроизводит его команды; ExperimentRunner создаёт независимый движок каждого прогона. Эти зависимости вынесены в подпись, чтобы не перегружать линии диаграммы.

Рис. 2: Основные классы ядра и иерархия канала уведомлений.

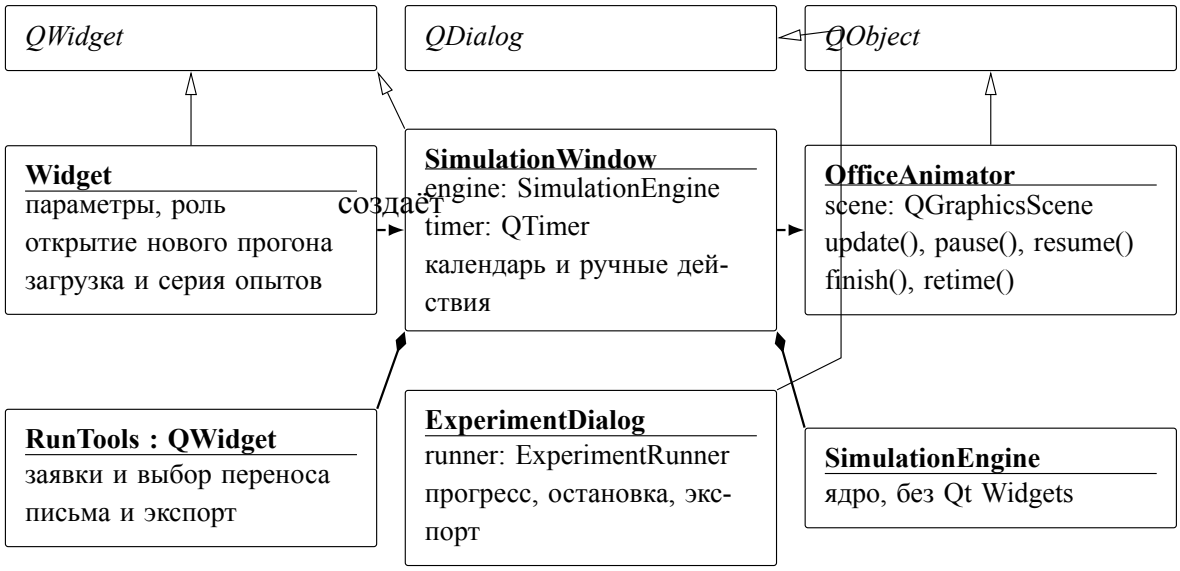


Рис. 3: Классы представления и их связь с моделью. Владение элементами интерфейса определяется родителями Qt.

2.2 Диаграмма объектов

Рисунок 4 показывает иллюстративный снимок до первого шага: начало 12 октября 2026 года, шесть отделов, по восемь сотрудников, включён регулярный календарь. Объекты ниже являются экземплярами классов проекта. Имена `engine`, `calendar` и `email` обозначают роли объектов; это не отдельные глобальные переменные программы. Из каталога показан только один отдел и один сотрудник.

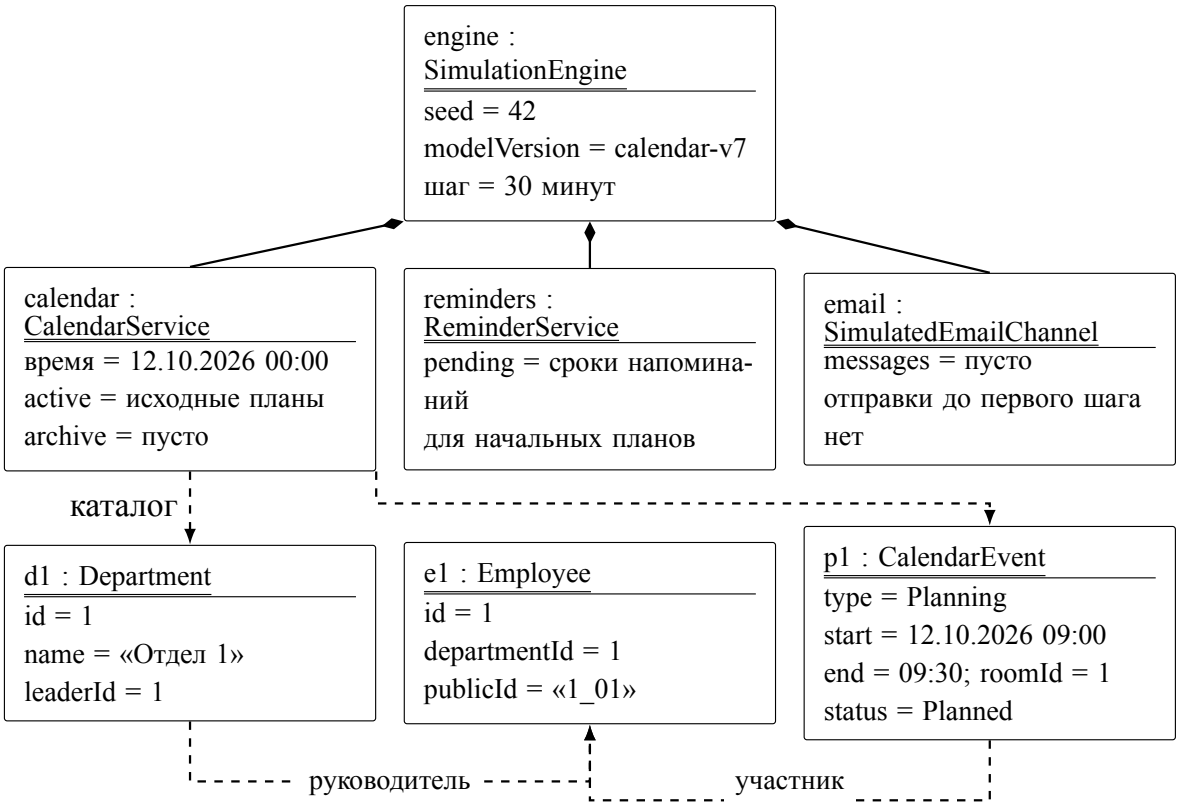


Рис. 4: Диаграмма объектов одного прогона (частичный снимок).

2.3 Правила модельного времени и актуализации

Ресурсы заняты на полуоткрытом интервале $[s_i, e_i)$. События i и j пересекаются, если

$$s_i < e_j \quad \wedge \quad s_j < e_i.$$

При пересечении общие участники или одинаковое непустое помещение создают конфликт. Касание границ, например 10:00–11:00 и 11:00–12:00, допустимо. Назначение в прошлое запрещено; произвольного пятичасового ожидания нет.

Состояния мероприятия: `Planned` → `InProgress` → `Completed`. Из `Planned` также возможна сервисная отмена с состоянием `Cancelled`. Завершённые и отменённые записи находятся в архиве.

Один шаг 30/60 минут последовательно обрабатывает каждую внутреннюю минуту:

1. Завершение событий и архивирование; старты на границе пока отложены.
2. В рабочее время — создание периодических и случайных заявок, попытки изменения расписания.
3. Отправка наступивших имитируемых напоминаний по актуальному календарю.
4. Запуск мероприятий на текущей границе и публикация истории.

На границе периода обрабатываются завершения, но новые заявки, уведомления и

старты уже не создаются. Событие, начавшееся до конца периода и продолжающееся после него, остаётся `InProgress`; будущий план — `Planned`. Они доступны в итоговом отчёте.

2.4 Подбор разрешений конфликта

Поиск сохраняет участников и длительность заявки и перебирает допустимые время и помещения. Используются рабочие дни в пределах заданного горизонта, начала на сетке 30 минут, исходное помещение, затем остальные помещения по ID. Каждый кандидат повторно проверяется календарным сервисом. Более раннее начало, чем исходно запрошенное, не предлагается. Поиск не переносит чужие мероприятия автоматически.

`Reject` фиксирует отказ. `Suggest` сохраняет несколько предложений до выбора пользователя. `AutoReschedule` применяет первый допустимый вариант немедленно. Если подходящих вариантов нет, заявка отклоняется. До принятия ожидающая заявка не резервирует ресурсы, поэтому вариант может устареть и требует повторной проверки.

3 Текстовые спецификации основных классов

3.1 Общие соглашения интерфейсов

Имена и сигнатуры ниже соответствуют заголовкам в `src/core` и `src/ui`. Возвращаемый `CalendarResult` содержит код `CalendarError`, пояснение, ID мероприятия, конфликтующие ID и ID заявки; `ok()` означает отсутствие ошибки. Отказ не подтверждает конфликтующее мероприятие. Некорректные настройки конструктора вызывают исключение `std::invalid_argument`; ошибки файлов и воспроизведения — `std::runtime_error`.

`EventId` — беззнаковый 64-битный идентификатор. Данные передаются преимущественно по константной ссылке; необязательные ссылки представлены `std::optional`. Все операции с модельным временем используют `QDateTime`.

3.2 EventData и CalendarEvent

Назначение. Представление пользовательских полей и управляемого сервисом состояния мероприятия. `EventData` хранит название, тип, начало и конец, необязательное помещение, ID участников, важность, организатора и необязательный ID шаблона повторения. `CalendarEvent` содержит `EventData`, ID, статус, две версии и снимки кодов и отделов участников.

Инварианты при принятии. Непустое название; существующие участники без повторов; непустой состав; существующий организатор; корректный интервал с концом после начала; допустимые перечисления; существующее помещение при его указании; отсутствие помещения у командировки. Начало не раньше текущего

времени. `version` меняется при редактировании, `scheduleVersion` — только при изменении времени, важности или состава участников. Клиент не задаёт ID и статус создаваемого события напрямую.

3.3 CalendarSetup

Назначение. Совокупность параметров одного опыта: каталог компании, период и шаг, рабочее время, генератор, начальные и периодические планы, политика конфликтов, правила напоминаний, роль и количество повторов. Структура не продвигает время и не содержит виджетов.

Проверка. `makeCompanyCatalog(setup)` создаёт каталог; конструктор движка проверяет согласованность параметров. Допустимы 5–9 отделов, 7–30 дней, шаг 30/60 минут, 1–1000 сотрудников на отдел. Рабочая неделя должна содержать хотя бы один рабочий день; начало рабочего времени раньше конца. Число участников не превышает штат. Интенсивности неотрицательны; веса важности имеют положительную сумму. Для включённого начального пресета требуется рабочий день минимум один час.

3.4 CalendarService

Назначение и состояние. Единственный владелец активного календаря, архива, истории и каталога одного прогона. Хранит текущий момент и конец горизонта. Не генерирует случайные события и не рисует интерфейс.

`CalendarService(catalog, start, horizonEnd).` **Вход:** Каталог и корректные границы времени.

Результат: Инициализированный пустой календарь.

Условия и эффекты: Начало строго раньше конца; ссылки каталога согласованы; коды сотрудников уникальны. Некорректная конфигурация вызывает исключение.

`checkEvent(data, actor, id=0).` **Вход:** Предлагаемые данные, контекст пользователя, необязательный ID редактируемого плана.

Результат: Результат проверки и список конфликтующих событий.

Условия и эффекты: Не изменяет состояние. Для id=0 проверяет создание, иначе существование, полномочия и статус редактируемого события.

`createEvent(data, actor).` **Вход:** Поля нового события и контекст пользователя.

Результат: Результат с новым EventId при успехе.

Условия и эффекты: Выполняет общую проверку; присваивает ID и версии, фиксирует снимки и запись Created. При допустимом начале в текущий момент событие может стартовать сразу, если старты не отложены движком.

`updateEvent(id, data, actor).` **Вход:** ID и новые поля существующего плана.

Результат: Успех либо причина отказа.

Условия и эффекты: Изменяется только Planned; проверяются права на исходные и новые данные. При успехе обновляются версии и история. Отказ сохраняет прежнее событие.

`calendar(filter).` **Вход:** Фильтр области, дат, часов, отдела, сотрудника, помещения и статуса.

Результат: CalendarQueryResult со списком и результатом проверки фильтра.

Условия и эффекты: Отбор по пересечению интервалов; просмотра достаточно для фильтрации, состояние не меняется.

`advanceTo(time, startAtBoundary=true).` **Вход:** Неубывающий момент внутри горизонта и флаг обработки стартов.

Результат: Результат продвижения времени.

Условия и эффекты: Обрабатывает все начала и окончания внутри интервала; завершения переносятся в архив. Режим false позволяет движку обработать заявки и уведомления до граничных стартов.

`cancelEvent(id, actor).` **Вход:** ID запланированного события и контекст.

Результат: Успех либо отказ.

Условия и эффекты: Проверяет полномочия и статус, переносит событие в архив как Cancelled. Метод есть в ядре, но отдельной кнопки отмены в UI нет.

Читающие операции. `event(id)` возвращает необязательное событие; `history(id)` — историю всех или одного события; `historySince(offset)` — её хвост; `participantCode` и `participantKey` преобразуют внутренние и публичные ID. `employeeBusy` проверяет участие в текущем мероприятии.

3.5 SimulationEngine

Назначение. Координация одного опыта. Содержит календарь, параметры, генератор `std::mt19937`, напоминания, почтовый канал, заявки, журнал и команды. От Qt Widgets не зависит.

`SimulationEngine(setup).` **Вход:** Параметры CalendarSetup.

Результат: Готовый к первому шагу движок.

Условия и эффекты: Проверяет параметры, создаёт каталог и исходные регулярные планы при включённом пресете. Версия алгоритма — calendar-v7.

`step()`. **Вход:** Явных параметров нет; используется сохранённый шаг.

Результат: CalendarResult.

Условия и эффекты: Продвигает время на 30/60 минут с обработкой внутренних минут, не переходя конец периода. После конца возвращает отказ. Одна реализация используется таймером, кнопкой шага и ExperimentRunner.

`reset()`. **Вход:** Нет.

Результат: Исходное состояние с теми же параметрами.

Условия и эффекты: Очищает историю, заявки, уведомления и команды, восстанавливает seed и начальный календарь. Это новый опыт, а не возврат к сохранённой точке.

`createEvent / updateEvent`. **Вход:** Данные мероприятия, контекст пользователя, при изменении ID.

Результат: Результат с requestId.

Условия и эффекты: Регистрирует команду и заявку, обращается к сервису, применяет выбранную политику конфликтов, публикует историю и обновляет план напоминаний.

`acceptSuggestion(requestId, index)`. **Вход:** ID ожидающей заявки и индекс варианта с нуля.

Результат: Результат новой попытки.

Условия и эффекты: Исходная заявка получает признак selected; выбранные данные повторно проходят общую проверку. Устаревший вариант не считается подтверждённым. Полномочия оператора в UI дополнительно проверяет RunTools.

`rejectSuggestion(requestId)`. **Вход:** ID ожидающей заявки.

Результат: Результат снятия ожидания.

Условия и эффекты: Не отменяет календарное мероприятие; завершает рассмотрение неподтверждённой заявки. Используется при возврате к редактированию.

`trigger(kind)`. **Вход:** Тип календарного сценария.

Результат: Результат сценарной заявки или переноса.

Условия и эффекты: Поддерживает пять типов мероприятий и изменение расписания. Некорректный тип и отсутствие подходящих планов приводят к отказу.

`replayCommand(command)`. **Вход:** Команда JSON с операцией, аргументами и временем.

Результат: Изменение восстановленного движка; при ошибке исключение.

Условия и эффекты: Время команды должно совпасть с текущим; неизвестная операция отклоняется. Читающие методы возвращают календарь, заявки, письма, журнал и команды без права изменения.

3.6 ConflictResolver

Назначение. Поиск вариантов без изменения календаря.

`alternatives(calendar, setup, requested, actor, id=0).` **Вход:** Текущий календарь, параметры поиска, исходная заявка, права и ID изменяемого события.

Результат: QVector<EventData> размером не больше maxSuggestions.

Условия и эффекты: Сохраняет длительность и участников, перебирает рабочие даты, начала и помещения. Не принимает конфликтующий кандидат. Пустой результат означает отсутствие найденного варианта в заданном окне, а не невозможность любого расписания вообще.

3.7 ReminderService и канал уведомлений

Состояние. `ReminderService` хранит для EventId версию расписания, первый и следующий сроки и период в секундах.

`observe(records, rules).` **Вход:** Новые записи истории и три правила по важности.

Результат: Обновлённый план уведомлений.

Условия и эффекты: Создаёт или перестраивает серию при новой версии; прекращает её после старта, завершения или отмены. Та же версия повторно не планируется.

`deliver(calendar, channel).` **Вход:** Текущий календарь и канал доставки.

Результат: Вызовы send для наступивших сроков каждому участнику.

Условия и эффекты: Уведомляет строго до начала мероприятия и до конца периода; передвигает курсор, предотвращая дубли. Позднее создание компенсирует пропущенное окно одной отправкой.

`NotificationChannel::send(message).` **Вход:** Значение EmailMessage с адресатом, событием, версией, текстом и временем.

Результат: Контракт виртуальной операции доставки.

Условия и эффекты: Абстрактный метод; планировщик не зависит от конкретного транспорта. Виртуальный деструктор обеспечивает корректное уничтожение через базовый тип.

`SimulatedEmailChannel::send(message).` **Вход:** Сформированное уведомление.

Результат: Запись в messages().

Условия и эффекты: deliveredAt приравнивается к sentAt, сообщение добавляется в журнал. Сеть не используется; прежние тексты сохраняются как снимки.

3.8 SimulationReport

`create(engine).` **Вход:** Движок в любом текущем состоянии.

Результат: QJsonObject полного отчёта.

Условия и эффекты: Не изменяет модель; содержит конфигурацию, версию, события всех статусов, историю, заявки, письма, журнал, сотрудников и отделы. До завершения отчёт промежуточный.

`metrics(engine).` **Вход:** Текущий движок.

Результат: QJsonObject счётчиков и занятости.

Условия и эффекты: Учитывает фактически начатые мероприятия; обрезает занятость прошедшим временем. Будущие бронирования в занятость не входят.

`csv(report) / summary(metrics).` **Вход:** Полный JSON-отчёт либо метрики.

Результат: CSV UTF-8 либо текстовая сводка.

Условия и эффекты: В общем CSV столбцы section,key,value; вложенные объекты сериализуются и экранируются. Плоский CSV серии опытов создаёт ExperimentRunner.

3.9 SimulationStore и JsonCodec

`snapshot / save.` **Вход:** Движок; при сохранении путь файла.

Результат: JSON состояния либо атомарно записанный файл.

Условия и эффекты: Содержит параметры, команды, контрольный календарь и SHA-256 отчёта вместе с состоянием генератора. Поддерживается формат 3 и только calendar-v7.

`restore / load.` **Вход:** JSON состояния либо путь к нему.

Результат: `std::unique_ptr<SimulationEngine>`.

Условия и эффекты: Проверяет формат и версию, создаёт новый движок, проигрывает команды и сравнивает контрольные данные. Ошибка не подменяет открытый прогон. Состояния прежних генераторов отклоняются.

`saveConfiguration / loadConfiguration.` **Вход:** Параметры и путь либо путь к JSON настроек.

Результат: Файл либо CalendarSetup.

Условия и эффекты: Параметры проверяются созданием движка; файл настроек не содержит накопленной истории. JsonCodec выполняет преобразования, проверки типов, дат и атомарную запись через QSaveFile.

3.10 ExperimentRunner

`ExperimentRunner(cases, repetitions).` **Вход:** Непустой список сценариев и 1–1000 повторов.

Результат: Готовая серия независимых опытов.

Условия и эффекты: Проверяет параметры сценариев и отсутствие переполнения seed+i.

`comparison(base).` **Вход:** Базовые параметры.

Результат: Шесть ExperimentCase.

Условия и эффекты: Три политики конфликтов при исходной и удвоенной интенсивности заявок. Остальные параметры сохраняются.

`advance().` **Вход:** Нет.

Результат: Признак оставшейся работы.

Условия и эффекты: Выполняет один шаг текущего движка; в пакетном Suggest после шага выбирает первый вариант. По завершении сохраняет метрики и переходит к следующему опыту.

`report() / csv().` **Вход:** Текущее состояние серии.

Результат: JSON с наблюдениями и агрегатами либо плоский CSV.

Условия и эффекты: Учитывает только завершённые опыты; незавершённая серия имеет complete=false. Для числовых метрик вычисляются среднее и выборочное стандартное отклонение.

3.11 Основные классы интерфейса

Widget : QWidget. Проверяет поля параметров, открывает новое `SimulationWindow`, загружает настройки или состояние, открывает серию опытов. Экземпляр прогона получает собственную копию параметров.

SimulationWindow : QWidget. Владеет движком и таймером; показывает календарь, архив и журнал. `restoreState(engine)` заменяет движок проверенным состоянием. Приватные операции добавления, редактирования и выбора доступны через сигналы виджетов; при открытии формы время останавливается. `simulationEngine()` и `calendarService()` предоставляют константный доступ для проверок.

RunTools : QWidget. Отображает заявки, письма, общий список событий и отчёт. Перед выбором предложения проверяет полномочия оператора; затем движок повторно проверяет расписание. Через переданные обратные вызовы ставит прогон на паузу, обновляет представление и открывает форму заявки.

OfficeAnimator : QObject. `update(calendar,durationMs)` перестраивает расположение и перемещение графических объектов; `pause`, `resume`, `finish`, `retime` управляют визуальным движением. Сцена владеет графическими элементами, родительские QObject — дочерними объектами Qt. Аниматор не меняет календарь.

4 Использованные инструментальные средства

Средство	Применение
C++17	Классы, структуры, наследование, композиция, enum class, RAII, std::optional, std::unique_ptr и стандартные распределения.
Qt Core	Даты и время, строки, контейнеры, JSON и атомарные файлы.
Qt Widgets	Окна, формы, вкладки, таблицы, календарные карточки, диалоги.
QGraphicsScene, QGraphicsView	Сцена офиса и её отображение; движение сотрудников реализуется через QVariantAnimation.
Qt Test и CTest	Автоматические проверки календаря, модели, сервисов и интерфейса; offscreen для GUI в тестовом окружении.
CMake	Описание библиотек, исполняемых целей и тестов; минимальная версия в проекте 3.16.
Qt Creator	Среда открытия и сборки CMake-проекта; локальные файлы конфигурации не являются обязательной частью исходников.
Git и GitHub	История исходного кода и удалённый репозиторий проекта.
Python 3	Проверка экспериментальных данных и генерация таблиц и текста результатов для LaTeX. Для этого скрипта внешние пакеты не требуются.
LaTeX, TikZ	Текст отчёта, таблицы и векторные диаграммы классов и объектов. Сборка XeLaTeX или Tectonic.

Для актуальной проверки и исследования использованы macOS на ARM64, Apple Clang 21.0.0, Qt 6.11.2, CMake 4.4.3 и Python 3.15.0. Это версии среды выполнения измерений, а не обязательные минимальные версии всех зависимостей. CMake допускает Qt 5 или Qt 6, но в рамках настоящего обновления проверка выполнена на Qt 6; новая совместимость Qt 5 отдельным запуском не подтверждалась.

Рабочая команда сборки в использованном окружении:

```
cmake --build build/reorganized -j2
ctest --test-dir build/reorganized --output-on-failure
```

Для чистой установки необходимо сначала создать каталог сборки командой `cmake -S . -B build/qt6`, при необходимости указав путь к Qt через `CMAKE_PREFIX_PATH`. Название локального каталога сборки не меняет устройство программы.

5 Файловая и модульная структура

```
CMakeLists.txt
README.md
src/
  main.cpp
  core/
    calendarevent.h
    calendarsetup.h/.cpp
    calendarservice.h/.cpp
    simulationengine.h/.cpp
    conflictresolver.h/.cpp
    reminderservice.h/.cpp
    jsoncodec.h/.cpp
    simulationstore.h/.cpp
    simulationreport.h/.cpp
    experimentrunner.h/.cpp
    experimentmain.cpp
  ui/
    widget.h/.cpp
    simulationwindow.h/.cpp
    officescene.h/.cpp
    runtools.h/.cpp
    experimentdialog.h/.cpp
tests/
  calendarservice_test.cpp
  simulationengine_test.cpp
  completion_test.cpp
  calendarwindow_test.cpp
  fixtures/
documentation/
  README.md
  MODEL.md
  ARCHITECTURE.md
  report/
    report.tex
    sections/
    figures/
    make_results.py
    results_table.tex
    results_text.tex
```

```
  data_manifest.json
experiments/
  results-v7/
  verification-v7/
```

Файл или группа	Ответственность
calendarevent.h	Структуры данных компании, событий, фильтров, истории, контекста и результата операций.
calendarsetup.*	Параметры опыта, создание исходного каталога, преобразование модельной даты и времени.
calendarservice.*	Проверки ресурсов и прав, активный календарь, архив, история, переходы статусов.
simulationengine.*	Начальный календарь, модельная минута, случайные сценарии, заявки и команды.
conflictresolver.*, reminderservice.*	Варианты переносов, планировщик уведомлений и имитируемый канал.
jsoncodec.*, simulationstore.*	JSON, проверка файлов, сохранение и контролируемое воспроизведение.
simulationreport.*	Метрики, полный JSON, CSV и текстовая сводка.
experimentrunner.*, experimentmain.cpp	Серии опытов, агрегирование и консольная точка входа.
src/ui	Только представление и управление через публичные операции ядра.
tests	Четыре набора автоматических проверок.
documentation/experiments/ results-v7	Фактические наблюдения текущего исследования, настройки, пример отчёта и сохранения.
documentation/experiments/ verification-v7	Логи сборки и проверки, JUnit XML и сведения о среде.

calendar_core зависит от Qt Core; calendar_ui — от ядра и Qt Widgets. Modeling запускает GUI через src/main.cpp. modeling_experiments работает с QCoreApplication и не требует дисплея. Локальные build/ и .qtcreator/ содержат служебные данные и исключаются из Git. Каталог documentation/experiments/results содержит исторические измерения v4 и не подменяет набор results-v7.

6 Краткая характеристика пользовательского интерфейса

6.1 Задание параметров

Начальное окно содержит группы «Компания и время», «Сценарий», «Периодические семинары», «Напоминания», «Конфликты и эксперимент». Задаются размер компании, период, шаг, рабочая неделя, интенсивности, участники, сроки, важность, роль и seed. Не относящиеся к выбранной политике поля поиска отключаются.

Доступны проверка параметров, возврат к значениям по умолчанию, сохранение и загрузка настроек, открытие состояния и кнопка «Провести серию экспериментов…». Кнопка «Открыть моделирование» создаёт независимый прогон с копией настроек. Изменение исходной формы не меняет параметры уже открытого движка.

6.2 Окно моделирования

В верхней части расположены запуск, пауза, следующий шаг, сброс, скорость и «+ Событие». Сводка показывает модельное время и количество активных и архивных событий. Значения скорости 0.5×, 1×, 2× и 5× регулируют таймер и анимацию.

Вкладка	Назначение
Журнал	Уже совершённые действия с модельным временем. Двойной щелчок открывает соответствующее мероприятие.
Календарь и архив	Просмотр карточек по датам, статусам, сотрудникам, отделам и часам внутри каждого дня. Завершённые мероприятия доступны в архиве.
Офис	План помещений, рабочие места и зоны встреч. Движущиеся объекты показывают переходы сотрудников между местами работы, мероприятиями и командировками.
Участники	Публичный ID, имя, отдел и текущая занятость; руководитель дополнительно обозначен.
Отделы	Название, число сотрудников и руководитель. Ручных кадровых кнопок нет.
Типы событий	Каталог пяти типов мероприятий и операции переноса с весами и длительностями.
Заявки, почта и отчёт	Рассмотрение конфликтов, имитируемые сообщения, полный список событий, сводка, сохранение и экспорт.

6.3 Добавление, редактирование и конфликты

Форма мероприятия содержит название, тип, начало и конец, помещение, участников, важность и организатора. Участники задаются публичными ID через запятую. После сохранения ядро проверяет данные, права, занятость и пересечения. Некорректный ввод не создаёт подтверждённое мероприятие.

Открытие формы ставит прогон на паузу. Запланированное событие редактируется только при наличии полномочий. Начатые, завершённые и недоступные по роли события показываются без возможности сохранения. Отдельной кнопки отмены мероприятия нет.

При `Suggest` новые ожидающие заявки останавливают автоматическое продвижение. На вкладке «Заявки» показываются причина и ID конфликтующих событий. Кнопка «Выбрать вариант…» открывает предложенные время и помещения; выбор проверяется повторно. «Редактировать заявку…» снимает ожидание старой попытки и открывает её поля для новой. Для изменения уже существующего конфликтующего мероприятия пользователь может открыть его карточку в календаре в пределах своей роли.

6.4 Почта, отчёт и исследование

Имитируемые сообщения объединяются для просмотра по рассылке; полный текст и адресаты открываются отдельно. В статистике доставка считается каждому адресату. Экспорт JSON/CSV доступен как до завершения, так и после него; промежуточный отчёт содержит текущее модельное время.

Окно серии экспериментов показывает число завершённых запусков и агрегаты стратегий. Остановленная серия отмечается незавершённой. В отличие от обычного окна, выбор в пакетном `Suggest` выполняет программное правило, поэтому серия не ждёт участия человека.

7 Проведённые эксперименты и исследование модели

7.1 Цель и схема исследования

Исследование выполнено на текущем генераторе `calendar-v7` 6 ноября 2026 года. Цель — сравнить актуализацию календаря, отказы и завершённые мероприятия при трёх способах обработки конфликтов и двух интенсивностях поступления заявок. Исходные данные получены запуском существующего консольного средства проекта, без изменения алгоритма ради результатов.

```
build/reorganized/modeling_experiments \
  --output documentation/experiments/results-v7 \
  --repetitions 20
```

План включает шесть сочетаний: `Reject`, `Suggest` и `AutoReschedule` при интенсивностях 6 и 12 случайных заявок на фирму за рабочий день. На сочетание выполнено 20 опытов с seed 42–61, всего 120 запусков. Для пакетного `Suggest` задан явный оператор: первый предложенный вариант принимается после каждого шага. В обычном интерфейсе такой выбор делает пользователь.

Параметр	Значение исследования
Начало и период	12.10.2026, 14 дней
Отделы и штат	6 отделов, по 8 сотрудников, всего 48
Помещения	6 комнат отделов и конференц-зал
Рабочее время и шаг	Пн–Пт, 09:00–18:00; 30 минут
Исходный календарь	60 планёрок и 2 совещания руководителей
Периодические семинары	Включены; период 7 дней, заблаговременность 2 дня, 14:00, 90 минут
Случайные заявки и изменения	6/12 заявок; 1 попытка изменения за рабочий день
Участники случайного плана	2–8; без повторов
Дальний срок планирования	1–5 рабочих дней; отдельная ветвь текущего дня
Веса важности	0.5; 0.35; 0.15
Напоминания	За 1/2/4 календарных дня, повторы каждые 24 часа
Поиск переносов	5 рабочих дней, максимум 3 варианта

7.2 Статистические факторы и показатели

За рабочую минуту число новых заявок распределено по Пуассону с параметром λ/M, где $M = 540$ — число минут рабочего дня, λ — заданная дневная интенсивность. Независимо формируются попытки изменения расписания. Тип, важность и допустимые участники выбираются по дискретным и равномерным распределениям; начало выбирается среди допустимых минут рабочего дня.

Основные показатели: созданные и завершённые мероприятия, конфликтные заявки, окончательно отказанные попытки, имитируемые доставки и доля занятости помещений. Будущие планы, выходящие за горизонт, могут быть созданы, но ещё не завершены к его концу. Поэтому число созданных мероприятий не равно числу завершённых.

Для $n = 20$ наблюдений показателя x вычисляются

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad s_x = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}}.$$

Доля занятости — сумма рабочих секунд фактически начавшихся событий, обрезанных прошедшим временем, делённая на прошедший рабочий фонд всех помещений. Будущие бронирования не включаются.

7.3 Фактические результаты

В таблице приведены средние значения и выборочные стандартные отклонения в формате $\bar{x} \pm s_x$. Обозначения: R — отклонение, S — предложения с выбором первого варианта после шага, A — немедленный автоматический перенос; λ — дневная интенсивность. Подробные наблюдения доступны в JSON и CSV набора `results-v7`.

Режим	λ	Завершено	Отказано	Занятость, %
R	6	93.05 ± 3.71	20.00 ± 4.12	9.46 ± 0.52
S	6	107.60 ± 5.78	0.05 ± 0.22	11.32 ± 0.82
A	6	107.55 ± 5.77	0.05 ± 0.22	11.31 ± 0.82
R	12	119.05 ± 5.03	50.80 ± 8.84	12.80 ± 0.91
S	12	153.45 ± 10.76	0.25 ± 0.64	17.42 ± 1.81
A	12	153.25 ± 10.60	0.20 ± 0.52	17.40 ± 1.77

Таблица 5: Результаты 120 прогонов calendar-v7: по 20 наблюдений на сочетание.

Интенсивность 6. При отклонении конфликтов завершено в среднем 93.05 мероприятий; при автоматическом переносе — 107.55. Разница составляет 14.50 мероприятия. Среднее число отказанных попыток соответственно 20.00 и 0.05; имитируемых доставок — 1229.70 и 1364.05.

Интенсивность 12. При отклонении конфликтов завершено в среднем 119.05 мероприятий; при автоматическом переносе — 153.25. Разница составляет 34.20 мероприятия. Среднее число отказанных попыток соответственно 50.80 и 0.20; имитируемых доставок — 1414.85 и 1755.25.

Наблюдение по стратегии предложений. Результаты Suggest с выбором первого варианта близки к немедленному автоматическому переносу. Это согласуется с заданным программным правилом решения, но не доказывает равенство стратегий при другом поведении оператора. Большее число конфликтных заявок при переносах не означает ухудшение проверки: более заполненный календарь создаёт дополнительные пересечения, которые могут завершаться успешным переносом.

7.4 Ограничения интерпретации

- Веса и длительности демонстрационные, эмпирическая калибровка по данным реальной фирмы не проводилась.

- При выборе Suggest возникает новая проверяемая попытка. Поэтому суммарное число заявок между стратегиями не является одним и тем же фиксированным внешним потоком.
- Одинаковый seed не обеспечивает одинаковую последовательность последующих событий между стратегиями: результаты операций меняют состояние и ветвление генерации.
- Эксперимент оценивает модель с автоматическим оператором Suggest, а не поведение реального пользователя.
- Значимость различий, причинный эффект и универсальное превосходство стратегии не устанавливались. Выводы относятся к указанным параметрам и выполненной серии.

Набор v4 в `documentation/experiments/results` является историческим. Его наблюдения не использованы в таблицах текущего отчёта. Скрипт `make_results.py` перепроверяет балансы, средние и стандартные отклонения v7 и фиксирует источник и контрольную сумму в `data_manifest.json`.

8 Подготовка Git-репозитория

В проекте настроен удалённый репозиторий:

`https://github.com/vladimirspanov-spec/Modeling`

Это адрес origin из локальной конфигурации Git. В рамках подготовки отчёта удалённое содержимое не проверялось и публикация изменений не выполнялась, поэтому совпадение GitHub с текущим рабочим деревом не заявляется.

Для сдачи в репозиторий должны входить исходники `src`, CMake, тесты, руководство, LaTeX-исходники и актуальный PDF, настройки исследования и результаты v7. Служебные каталоги сборки, локальные настройки IDE и секреты не являются частью сдаваемого комплекта. Перед публикацией необходимо проверить состав изменений, сохранить их коммитом и отдельно опубликовать в origin. Наличие настроенного адреса само по себе не означает выполнение требования о полном актуальном репозитории на GitHub.

A Проверка реализации и воспроизводимость

A.1 Автоматические проверки

В актуальном окружении выполнена сборка и запуск всех четырёх целей CTest. Результат: 4 из 4 наборов прошли. Это относится к совокупности наборов, а не к числу отдельных тестовых случаев.

Цель	Проверяемые свойства
`calendarservice`	Входные данные, права, конфликты ресурсов, интервалы и границы, фильтры, жизненный цикл и архив.
`simulationengine`	Ежедневные планёрки всех отделов, семинары, состав руководителей, короткий срок планирования, генерация и независимость от шага.
`completion`	Напоминания, дедупликация и версии, сохранение/продолжение, метрики, экспериментальные агрегаты, явный отказ для прежних генераторов.
`calendarwindow`	Передача параметров, календарные формы, права ролей, ручной выбор переноса, возврат к заявке, фильтры, пауза и визуальное движение.

Логи и JUnit-результат сохранены в каталоге:

`documentation/experiments/verification-v7`

Дополнительно скрипт генерации таблиц повторно вычисляет показатели исследования из отдельных прогонов. Успешные автоматические проверки подтверждают проверенные случаи, но не заменяют проверку всех возможных пользовательских сценариев и согласование требований преподавателем.

A.2 Воспроизведение исследования и сборка отчёта

Для повторения серии нужны та же версия алгоритма и совместимая реализация стандартных случайных распределений C++. Разные стандартные библиотеки могут дать другую случайную последовательность даже при одном seed; идентичность между платформами не обещается.

```
cmake -S . -B build/qt6 -DBUILD_TESTING=ON
cmake --build build/qt6 -j2
ctest --test-dir build/qt6 --output-on-failure
./build/qt6/modeling_experiments \
  --output documentation/experiments/results-v7 \
  --repetitions 20
python3 documentation/report/make_results.py
cd documentation/report
xelatex -interaction=nonstopmode -halt-on-error report.tex
xelatex -interaction=nonstopmode -halt-on-error report.tex
```

Альтернатива двум вызовам XeLaTeX — `tectonic report.tex`. Для диаграмм используется TikZ, для кириллицы — fontspec и babel. Основной шрифт — DejaVu Serif; если он отсутствует, используется Times New Roman. Аналогично предусмотрены запасные системные шрифты для гротеска и моноширинного текста.

А.3 Соответствие пунктам отчёта из задания

Требование	Раздел
Уточнённая постановка задачи	1
Диаграмма классов	2
Диаграмма объектов в рамках проектирования	2
Текстовые спецификации основных классов	3
Инструментальные средства	4
Файловая структура	5
Краткая характеристика интерфейса	6
Описание проведённых экспериментов	7
Подготовка полного Git-репозитория на GitHub	8: указан адрес и фактический статус публикации

Открытые вопросы: согласование имитации электронной почты и публикация актуального комплекта на GitHub. Эти действия не подменены описанием и не заявлены выполненными.